

Gosonic Workflow Architecture Canon v1

1. Foundational Principle

Gosonic is not a collection of call flows.

Gosonic is an operational communication infrastructure platform.

Every phone call, message, booking, dispatch action, acknowledgement, escalation, and resolution should be treated as part of a larger operational workflow.

The call is only the entry point.

The platform must understand what happened, what state the work is in, what still needs to happen, who or what is responsible for the next step, and whether the workflow completed successfully.

2. Core Architecture Rule

Gosonic should be built around events and workflow state, not around screens.

The UI should display operational truth.

The backend should own operational truth.

The database should preserve operational truth.

The voice agent should generate operational events, not act as the entire system.

3. Canonical Object Model

Client

■■■ Call

■■■ Workflow Instance

■■■ Operational Events

■■■ Workflow Steps

■■■ Current State

■■■ Timeline

A call may start a workflow.

A workflow may continue after the call ends.

A workflow may include SMS, booking, dispatch, acknowledgement, CRM sync, technician assignment, and resolution.

4. Canonical Workflow Lifecycle

received

→ intake_started

→ intake_completed

→ triage_completed

→ workflow_created

→ notification_sent

→ acknowledgement_pending

→ acknowledged

→ booking_pending

→ booked

→ dispatch_pending
→ dispatched
→ technician_assigned
→ service_in_progress
→ resolved
→ closed

Not every plan uses every stage.

Lite may stop at notification_sent.

Pro may continue to booked.

Pro+ may continue through dispatch and technician assignment.

Enterprise may extend into CRM, SLA tracking, multi-location routing, and escalation policy handling.

5. Workflow Instance

A workflow_instance represents the operational job created from a call or communication event.

Example:

A homeowner calls about no heat.

The Retell agent captures the issue.

The backend classifies it as urgent.

An SMS is sent to the business.

A workflow instance is created.

The workflow is now in notification_sent or acknowledgement_pending.

The call is complete.

The workflow may not be complete.

That distinction is foundational.

6. Operational Events

Every meaningful system action should produce an immutable event.

Examples:

call.started

call.ended

call.analyzed

intake.completed

triage.completed

workflow.created

sms.business.sent

sms.caller.sent

booking.requested

booking.completed

dispatch.requested

operator.acknowledged

technician.assigned

workflow.resolved

workflow.closed

workflow.failed

Events should not be overwritten.

They become the operational history.

The UI timeline should be derived from these events.

7. Workflow State

The workflow state is the current operational position of the job.

Examples:

new

in_progress

waiting_for_acknowledgement

waiting_for_booking

booked

dispatched

escalated

failed

resolved

closed

Events are history.

State is the current truth.

Both are needed.

8. Plan Capability Layers

Lite

Lite includes:

voice intake

triage

call summary

business SMS

optional caller SMS

call record

workflow record

basic timeline

Typical terminal state:

notification_sent

Pro

Pro adds:

calendar availability

booking request

booking confirmation

CRM/contact sync

customer confirmation

workflow booking state

Typical terminal state:

booked

Pro+

Pro+ adds:

dispatch workflow

operator acknowledgement

technician assignment

dispatch status

completion tracking

escalation handling

Typical terminal state:

resolved

Enterprise Direction

Enterprise extends the same architecture with:

multi-location routing

role-based permissions

SLA tracking

advanced escalation rules

analytics

audit logs

external system orchestration

custom workflow definitions

No separate architecture should be created for Enterprise.

Only additional capabilities should be enabled.

9. Database Direction

The next durable backend layer should eventually include:

clients

calls

client_settings

workflow_instances

operational_events

workflow_steps

workflow_transitions

workflow_assignments

workflow_notes

notifications

booking_records

dispatch_records

Immediate next tables should likely be:

workflow_instances

operational_events

Those two create the foundation.

Everything else can come later.

10. UI Direction

The Client Admin should eventually show:

Calls

Workflows

Timeline

Notifications

Bookings

Dispatch

Client Settings

Billing

The Calls view should not carry the whole product.

Calls are one operational surface.

The more important future surface is:

Workflow Detail

That view should show:

intake status

triage status

notification status

acknowledgement status

booking status

dispatch status

resolution status

event timeline

raw call reference

11. Color and Status Canon

Operational colors remain fixed:

green = successful progression / completed operational action

red = escalation / failure / intervention required

gray = infrastructure / persisted system state

white = pending / inactive / undefined

pink = brand accent only

Pink must not represent urgency, selection, completion, error, success, or workflow state.

12. Engineering Rule

Every new feature should answer these questions before implementation:

What event does this create?

What workflow state can it change?

What table owns the truth?

What UI surface displays it?

What plan tier enables it?

What happens if it fails?

Is it reusable across verticals?

If a feature cannot answer these questions, it is not ready to be built.

13. Immediate Next Build Sequence

The correct next engineering sequence is:

1. Fix duration mapping.
2. Add workflow_instances table.
3. Add operational_events table.
4. When a call is analyzed, create or update workflow instance.
5. Insert canonical events from call lifecycle.
6. Return workflow data through /calls or future /workflows endpoint.
7. Display workflow timeline inside expanded call detail.
8. Later separate Workflows into its own main view.

This keeps us disciplined.

No overbuilding.

No fake dashboard features.

No disconnected UI.

14. Canonical Statement

Gosonic workflows are event-driven operational lifecycles initiated by communication activity.

A voice call may begin the workflow, but the platform exists to track, coordinate, escalate, and complete the work that follows.

The product should scale from simple intake notification to full operational orchestration without changing its core architecture.

That is the canon.